MATRIXLANG · EXACTREDUCE

Proof-Guided Floating-Point Optimization

Exact Reassociation, Error Bounds, and Compiler Certificates

PRESENTED BY

A Aswanth Raj

GUIDE

Dr. Ranjithkumar S

School of Computer Science and Engineering, VIT Vellore



github.com/aswanth-07/matrixlang

Compilers make you choose: safe or fast

STRICT

1×

Same bits as the source,
but sums stay sequential

We want

both

and to know which
guarantee each output keeps

FAST-MATH

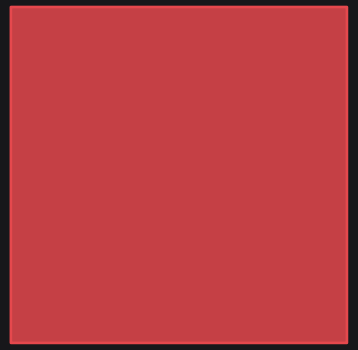
5–10×

Vectorized, but no one knows
which results changed

One matrix chain, two orders

$(A \cdot B) \cdot C$

intermediate



A·B is 100 × 100

69,800

scalar operations

$A \cdot (B \cdot C)$

intermediate



B·C is 2 × 2

1,396

scalar operations

A, C are 100 × 2, B is 2 × 100. **The reorder is 50× cheaper. Is it allowed? That depends on the values, not the program text.**

Three guarantees, three contracts

	Bit-identical same bits *e.g. 8-bit integers*	**Bound-preserving** same error bound *e.g. larger integers*	**Relaxed** true over the reals only *e.g. unbounded reals*
Strict default	admitted	—	—
Bounded	admitted	admitted	—
Algebraic ≈ fast-math	admitted	admitted	admitted

Each output is labelled with the weakest guarantee among the rewrites that reach it.

From value facts to certified outputs

MatrixLang

declared input domains

```
input(int8),
input(real(1))
```

ExactReduce

LLVM 18 pass on plain C

```
integer → double
conversions
```



Facts

grid + magnitude of every value

Obligations

exact in 53 bits? bound still holds?

Contract

is this level admitted?

Rewrite + certificate

reorder, vectorize, label output

Keep source order

strict fallback, nothing lost

A magnitude bound alone is not enough

COUNTEREXAMPLE

```
q = 2^-1074   a = 3q   b = ½   c = ¾

(a·b)·c   →   2q
a·(b·c)   →    q
exact     =   9q/8
```

Values inside [−1, 1], yet both orders break the claimed error bound: they underflow inexactly.

FIX

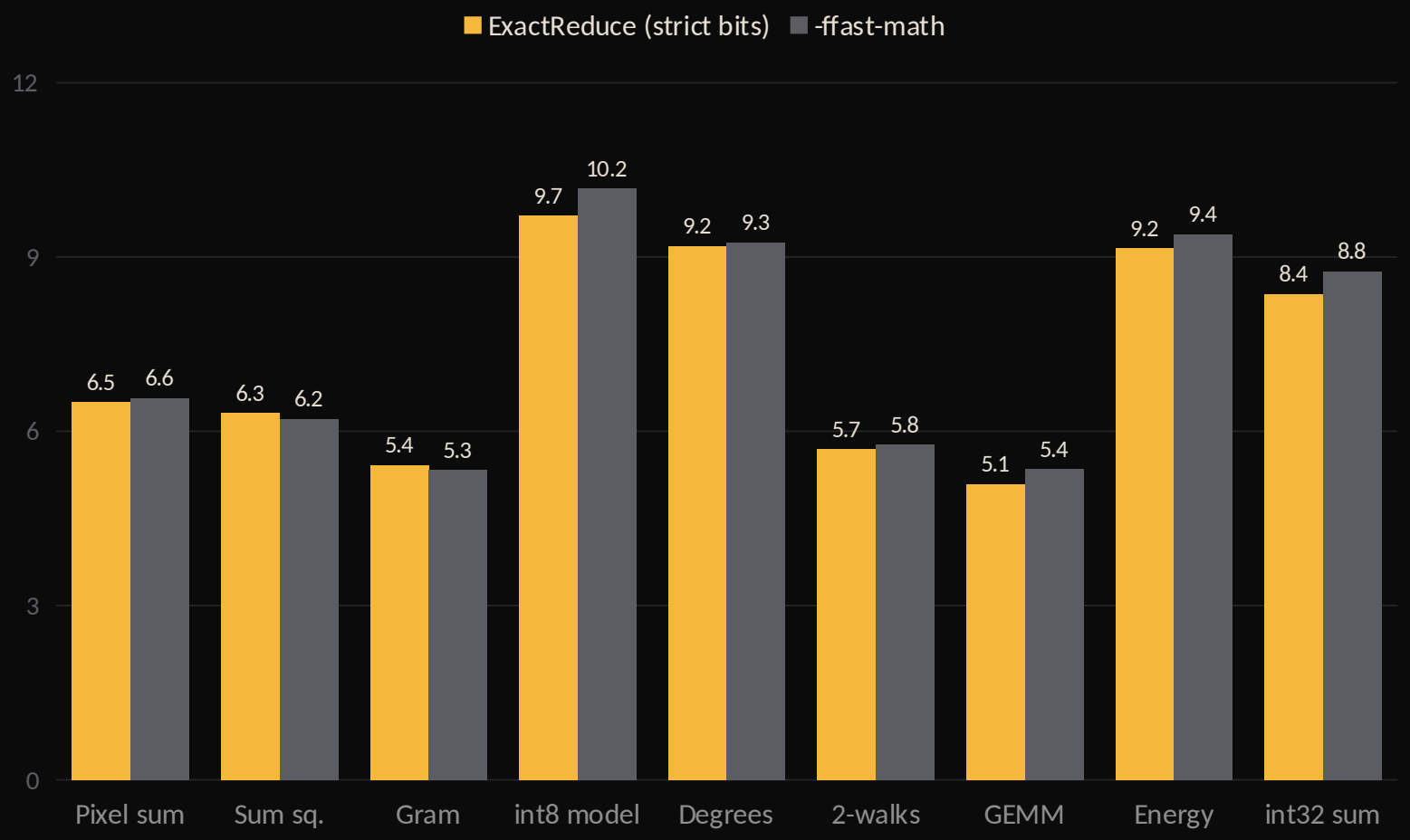
Also require a representable product grid

100% of the optimization kept on integer data: unchanged

43.7% kept on real(1) data: the conservative price

All 15,000 + 4,800 study programs replayed after the fix: zero failures.

Fast-math speed, strict results



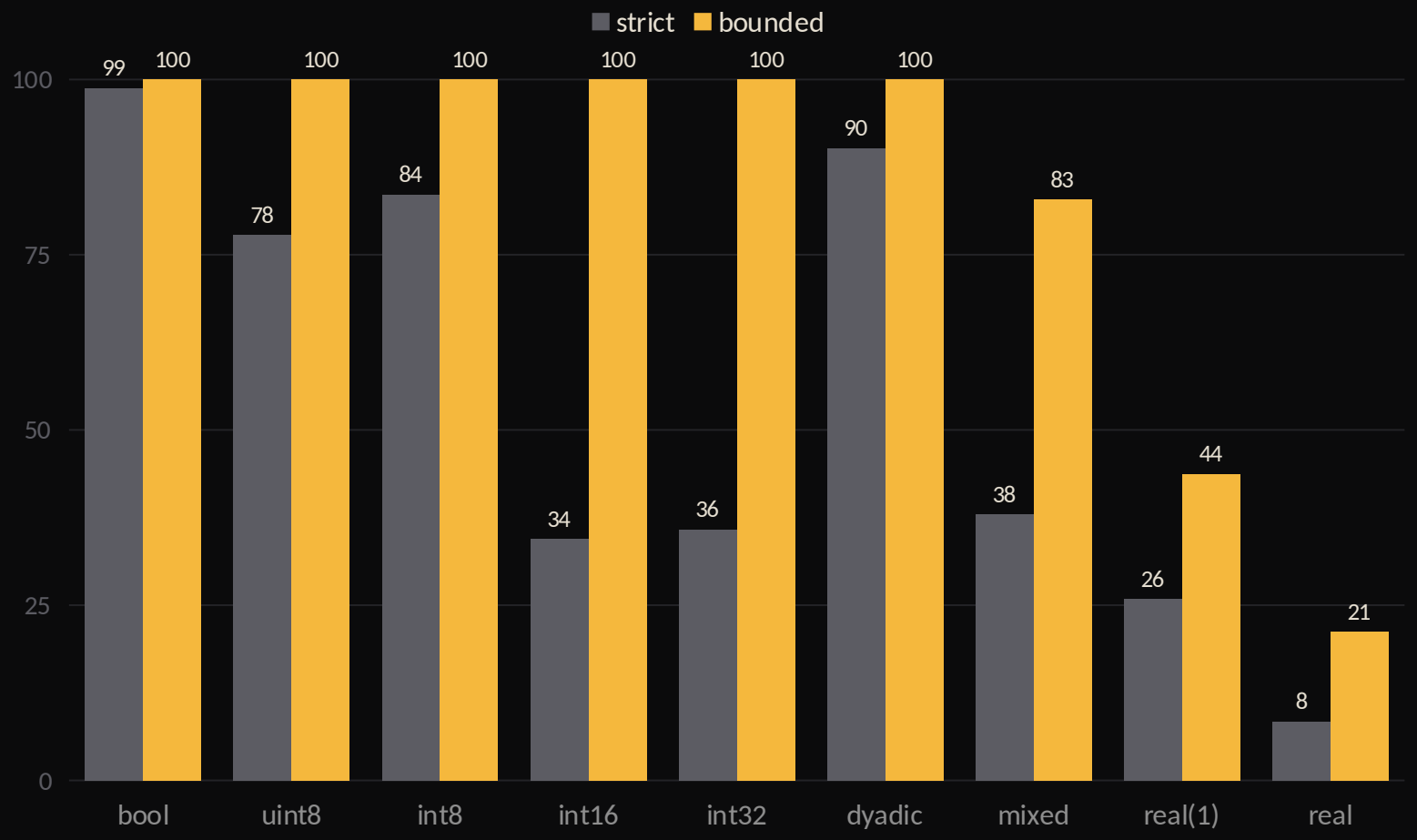
9.71×

up to, over strict Clang (5.09× at least) on nine integer-data kernels

95–102%

of fast-math's speedup, with exactly the strict result bits

Bounded mode keeps what strict must skip



100%

of the full algebraic saving, kept by bounded mode on every integer domain

15,000

generated programs, 60,000 compilations, none failing

Checked, not assumed

0

wrong outputs in 57,600 runs

every output certified bit-identical matched the unoptimized program

0

changes in 784 attack cases

inputs built to break the proof; fast-math changed 14

262

certificates replayed

in exact rational arithmetic by a separate checker, 0 failures

Where it helps, and what it costs

23 / 109

reductions sped up in four real code bases (CPython audioop, Pillow)

404 / 404

identical audioop results with and without the pass

8.05×

faster average of 8-bit audio samples

LIMITS

+29% compile time, +40% code size

from loop versioning

Bit-identical speedups need integer data

real-valued sums are declined

The proof covers regrouping only

the LLVM pass is tested, not verified

TAKEAWAY

Fast exactly where it is provably safe

A guarantee for every rewrite, a certificate for every output.

Thank you · Questions?



github.com/aswanth-07/matrixlang